超声波避障

超声波避障
设备连接
硬件连接
软件连接
实现思路
代码分析
实验效果

通过按键K1控制小车超声波避障功能的开启和关闭。

> 由于前面的教程已经讲解了所有小车模块的基础知识，小车控制教程不再重复讲解，主要讲解小车功能实现思路！

设备连接

硬件连接

使用Type-B数据线连接Arduino Uno和电脑。

软件连接

打开“Arduino IDE”软件，选择开发板对应的型号和串口号。

实现思路

通过超声波模块获取前面物体的距离，并根据距离信息控制小车运动。

按键K1：开启或关闭小车避障功能；

超声波模块：获取小车前方物体距离；

小车控制：

小车前方物体小于设定的距离（15cm）：小车后退；

小车前方物体处于设定的距离区间（15-30cm）：小车停止；

小车前方物体大于设定的距离区间（30cm）：小车前进。

代码分析

此处只简单介绍代码内容，详细代码建议参考对应代码文件，代码文件提供在下载专区！

- 包含`Wire`、`Adafruit_PWMServoDriver`库

```
#include <Wire.h>                    // 包含Wire(I2C)通讯库 Include Wire library
#include <Adafruit_PWMServoDriver.h>  // 包含Adafruit PWMServoDriver库 Include
Adafruit PWMServoDriver library
```

- 定义电机、按键、超声波控制引脚、I2C地址、电机频率以及初始速度

```
// 定义电机控制引脚 Define motor control pins
```

```
// 定义电机控制引脚 Define motor control pins
#define Motor_L1_F_PIN 11  // 控制小车左前方电机前进 Control the motor on the left front of the car
#define Motor_L1_B_PIN 10  // 控制小车左前方电机后退 Control the motor back on the left front of the car
#define Motor_L2_F_PIN 8   // 控制小车左后方电机前进 Control car left rear motor forward
#define Motor_L2_B_PIN 9   // 控制小车左后方电机后退 Control the car left rear motor back
#define Motor_R1_F_PIN 13  // 控制小车右前方电机前进 Control the right front motor of the car to move forward
#define Motor_R1_B_PIN 12  // 控制小车右前方电机后退 Control the motor back on the right front of the car
#define Motor_R2_F_PIN 14  // 控制小车右后方电机前进 Control car right rear motor forward
#define Motor_R2_B_PIN 15  // 控制小车右后方电机后退 Control car right rear motor back

// 定义底层驱动芯片参数 Bottom-layer driver chip related parameters
#define Bottom_Layer_Driver_ADDR 0x40

// 定义PWM频率 Define PWM frequency
#define PWM_FREQUENCY 50

// 定义按键引脚和控制状态 Define pin and key(button) states
#define KEY_PIN 7
#define Press_KEY 0
#define Release_KEY 1

// 定义超声波模块控制引脚 Define ultrasonic control pins
#define TRIG_PIN 11
#define ECHO_PIN 12

bool bCar_Switch = false;

int iCarSpeed = 50;
float Distance = 0.0;
const float FarDistance = 30.0;
const float NearDistance = 15.0;
```

- 枚举小车状态

```
// 枚举全向小车的常见运动方式 Enumerate the common movement modes of omnidirectional cars
enum OmniDirectionalCar {
  STOP,
  FORWARD,
  BACKWARD,
  LEFT,
  RIGHT,
  LEFT_ROTATE,
  RIGHT_ROTATE,
  LEFT_FORWARD,
  RIGHT_BACKWARD,
  RIGHT_FORWARD,
  LEFT_BACKWARD,
```

```
};
```

- 创建`Adafruit_PWMServoDriver`的实例

```
// 创建Adafruit_PWMServoDriver类的实例 Create an instance of the
Adafruit_PWMServoDriver class
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(Bottom_Layer_Driver_ADDR);
```

- 设置电机速度

```
/**
 * @brief 设置单个电机速度 Setting the Motor Speed
 * @param motor_forward_pin: 控制电机前进引脚 Control the motor forward pin
 * @param motor_backward_pin: 控制电机后退引脚 Control the motor backward pin
 * @param motor_speed: 设置电机速度 Setting the Motor Speed
 * @retval 无 None
 */
void setMotorSpeed(uint16_t motor_forward_pin, uint16_t motor_backward_pin, int
motor_speed) {
  motor_speed = map(motor_speed, -255, 255, -4095, 4095);
  if (motor_speed >= 0) {
    pwm.setPWM(motor_forward_pin, 0, motor_speed);
    pwm.setPWM(motor_backward_pin, 0, 0);
  } else if (motor_speed < 0) {
    pwm.setPWM(motor_forward_pin, 0, 0);
    pwm.setPWM(motor_backward_pin, 0, -(motor_speed));
  }
}
```

- 设置小车状态

```
/**
 * @brief 设置小车运动方式和速度 Set the car movement mode and speed
 * @param Movement: 小车运动方式 Car movement
 * @param Speed: 小车运动速度 Car speed
 * @retval 无 None
 */
void setCarMove(uint8_t Movement, int Speed) {
  switch (Movement) {
    case STOP:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
      break;
    case FORWARD:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
      break;
    case BACKWARD:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
```

```
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
    break;
  case LEFT:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
    break;
  case RIGHT:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
    break;
  case LEFT_ROTATE:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
    break;
  case RIGHT_ROTATE:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
    break;
  case LEFT_FORWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
    break;
  case RIGHT_BACKWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
    break;
  case RIGHT_FORWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
    break;
  case LEFT_BACKWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
    break;
  default:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
```

```
      break;
  }
}
```

- 获取按键状态

```
/**
 * @brief 获取按键状态 Get key(button) status
 * @param pin: 按键控制引脚 Control key(button) pins
 * @retval 按键状态 Key(button) Status
 */
int getKeyState(uint8_t pin) {
  if (digitalRead(pin) == LOW) {
    delay(20);
    if (digitalRead(pin) == LOW) {
      while (digitalRead(pin) == LOW)
        ;
      return Press_KEY;
    }
    return Release_KEY;
  } else {
    return Release_KEY;
  }
}
```

- 获取距离

```
/**
 * @brief 获取超声波距离 Get ultrasonic distance
 * @param trigPin: 触发测距引脚 Trigger pin
 * @param echoPin: 接收测距引脚 Echo pin
 * @retval 转换的距离 Measured distance (cm)
 */
float getDistance(int trigPin, int echoPin) {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH);
  float distance = (duration * 0.034 / 2);
  return distance;
}
```

- 小车避障功能

```
/**
 * @brief 小车避障功能 Car obstacle avoidance function
 * @param 无 None
 * @retval 无 None
 */
void Car_Avoid() {
```

```
  Distance = getDistance(TRIG_PIN, ECHO_PIN);
  delay(10);
  if (Distance < NearDistance) {
    setCarMove(BACKWARD, iCarSpeed);
  } else if (NearDistance <= Distance && Distance <= FarDistance) {
    setCarMove(RIGHT_ROTATE, iCarSpeed);
  } else if (FarDistance < Distance) {
    setCarMove(FORWARD, iCarSpeed);
  } else {
    setCarMove(STOP, iCarSpeed);
  }
}
```

- 设置小车功能开关

```
/**
 * @brief 设置小车功能开关 Set the car function switch
 * @param 无 None
 * @retval 开启/关闭 true/false
 */
bool setCarSwitch() {
  if (getKeyState(KEY_PIN) == Press_KEY) {
    bCar_Switch = !bCar_Switch;
  }
  return bCar_Switch;
}
```

- 初始化代码

```
void setup() {
  Wire.begin();                    // 初始化I2C通讯 Initialize I2C communication
  delay(1000);                     // 如果小车功能异常，可以增加这个延时 If the function
is abnormal, you can increase the delay
  pwm.begin();                     // PWM初始化 Initialize the Pulse Width
Modulation (PWM) library
  pwm.setPWMFreq(PWM_FREQUENCY);  // 设置PWM频率 Set the PWM frequency
  setCarMove(STOP, 0);             // 设置小车停止状态 Set the car to stop state
}
```

- 循环运行代码

```
void loop() {
  // 按键控制小车避障功能启停 The key control car obstacle avoidance function start
and stop
  if (setCarSwitch()) {
    Car_Avoid();
  } else {
    setCarMove(STOP, 0);
  }
}
```

实验效果

编译程序成功后，上传代码到Arduino Uno开发板。

程序启动后，按下K1按键，可以启动和关闭小车超声波避障功能。

实际使用：我们可以将小车放在平坦的地面上，按下K1按键启动小车功能，然后可以观察整个避障效果，若需要停止避障功能，可以按下K1按键关闭小车避障功能。

> 驱动电机需要外接电池组并且打开拓展板开关才可以正常驱动。
> 烧录程序不可以使用别的程序占用串口或者外接串口通讯模块（例如：`WiFi`摄像头模块），不然无法烧录程序或者提示错误信息！